

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
CENTRO EL NARANJO, MIXCO
FACULTAD DE INGENIERÍA EN SISTEMAS Y CIENCIAS DE LA COMPUTACIÓN
PROGRAMACIÓN III



Nombre:

Ruben Daniel Hernández González

Carnet:

9490-24-645

Guatemala, 30 de mayo de 2026

ESQUEMA CONCEPTUAL DEL SISTEMA

El sistema es un juego de Totito (Tic Tac Toe) que integra:

- Interfaz por consola (menú principal).
- Lógica del juego.
- IA basada en pesos (aprendizaje).
- Entrenamiento automático por simulación.
- Historial de partidas.
- Generación de gráficos de jugadas.

El sistema funciona a modo de ciclo: **Jugar - Registrar partidas - Entrenar IA - Mejorar decisiones.**

ARQUITECTURA DEL SISTEMA

El sistema está dividido en módulos independientes:

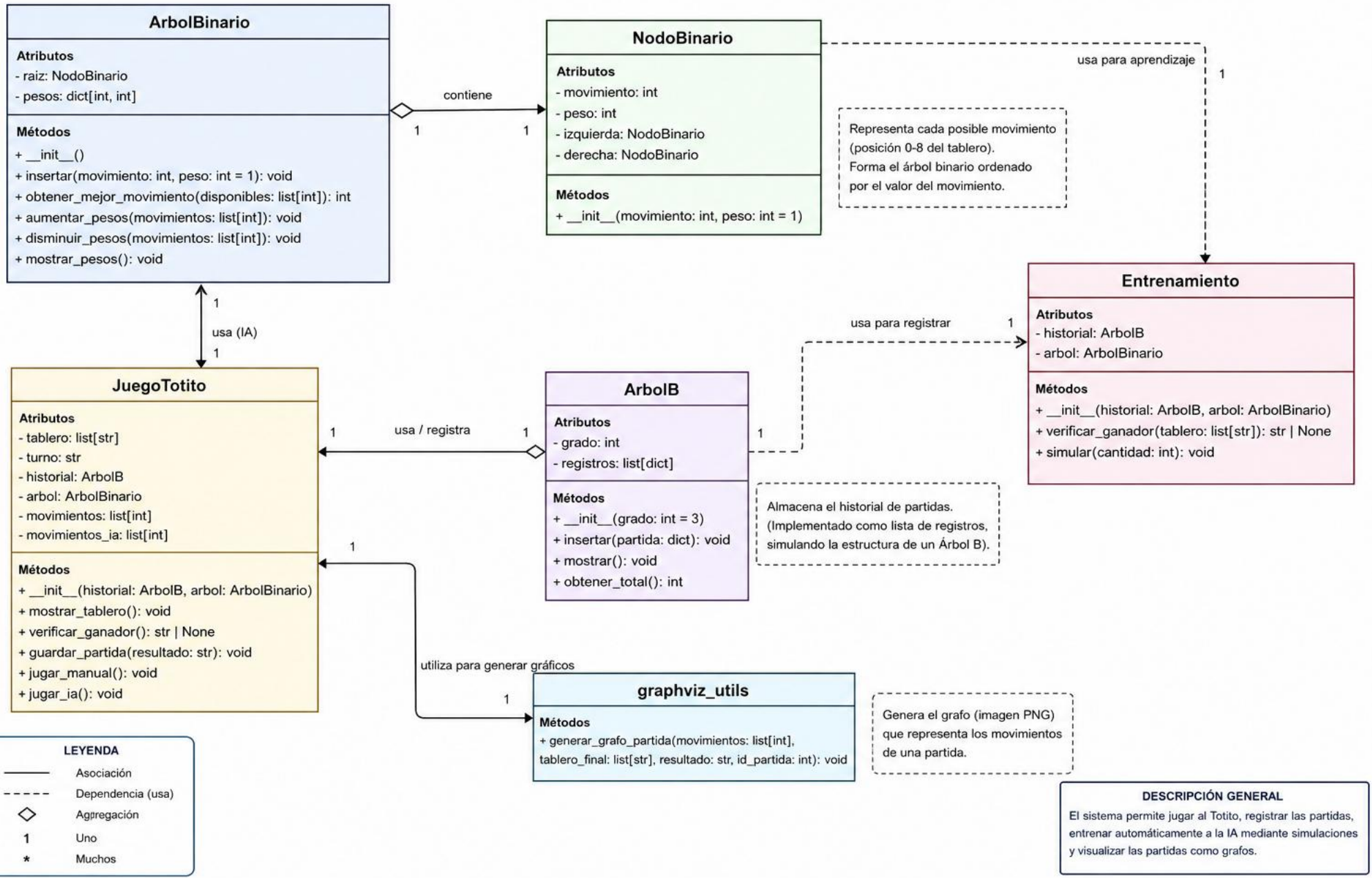
Componentes principales

- main.py - Control general del sistema.
- juego.py - Lógica del juego.
- arbol_binario.py - IA (pesos y decisiones).
- aprendizaje.py - Entrenamiento automático.
- arbol_b.py - Historial de partidas.
- graphviz_utils.py - Visualización de partidas.

Flujo general del sistema

1. Usuario entra al menú (main.py).
2. Se selecciona modo de juego.
3. Se ejecuta partida (juego.py).
4. Se guarda historial (arbol_b.py).
5. Se actualizan pesos de IA (arbol_binario.py).
6. Se puede entrenar automáticamente (aprendizaje.py).
7. Se generan gráficos (graphviz_utils.py)

DIAGRAMA DE CLASES – TOTITO INTELIGENTE



DESCRIPCIÓN DE CLASES

arbolBinario (IA principal)

Responsabilidad:

Gestionar la inteligencia del sistema mediante pesos.

Funciones clave:

- Selección de mejor movimiento.
- Ajuste de aprendizaje.
- control de nodos del árbol.

Importancia:

Es el “cerebro” del juego.

nodoBinario

Responsabilidad:

Representa cada movimiento posible.

Contenido:

- Posición del tablero.
- Peso.
- Conexiones izquierda/derecha.

juegoTotito

Responsabilidad:

Controla toda la partida.

Funciones:

- Manejar turnos.
- verificar ganador.
- Guardar partidas.

- interactuar con IA o jugador.

entrenamiento

Responsabilidad:

Generar partidas automáticas para entrenar la IA.

Funciones:

- simular juegos.
- ajustar pesos según resultados.

arbolB (Historial)

Responsabilidad:

Guardar historial de partidas.

graphviz utils

Responsabilidad:

Visualizar partidas como grafos.

ESTRUCTURAS DE DATOS UTILIZADAS

Árbol binario

Propósito:

Representar decisiones del juego.

Implementación:

Nodo con hijos izquierda/derecha.

Justificación:

Permite estructurar decisiones y asociar pesos a movimientos, es basicamente el “esqueleto” del sistema.

Listas

Uso:

- tablero del juego
- movimientos
- historial

Justificación:

Simple, rápida y eficiente para almacenar estados.

Árbol B (simulado)

Propósito:

Guardar historial de partidas.

Implementación:

Lista de diccionarios.

Justificación:

Permite registro estructurado de partidas sin complejidad.

Diccionario de pesos

Propósito:

Representar “memoria” de la IA.

Implementación:

`self.pesos = {0:1, 1:1, ...}`

Justificación:

Acceso rápido para decidir jugadas.

LÓGICA DEL APRENDIZAJE

El sistema usa un aprendizaje por refuerzo simple y simulación automática.

Cómo aprende

Cuando la IA gana, aumentan pesos y refuerza decisiones correctas. Cuando la IA pierde disminuyen los pesos y penaliza decisiones malas.

Evolución del sistema

Con más partidas:

- movimientos buenos – más peso
- movimientos malos – menos peso

DIAGRAMA DE FLUJO

TOTITO

